

技术报告 · 性能测试

PiLore 上下文缓存 利用率测试报告

三次模拟真实用户场景的 512K 上下文压测，验证追加式
persona 上下文对前缀缓存命中率与成本的影响

PiLore Team

V1.0 · 2026.08

基于 DeepSeek API 实测数据

| 目录

01	执行摘要	03
02	背景与问题定义	04
03	方法与发现	05
04	结论与建议	07
05	附录	08

01 · EXECUTIVE SUMMARY

执行摘要

PiLore 在保持真实用户使用模式的前提下，把 persona 方法论从「换入 systemPrompt」改为「追加式内部上下文」，512K 长对话下的累计缓存命中率从 **84.26%** 提升至 **97.11%**，估算成本从 **\$0.58** 降至 **\$0.12**，同时角色切换不再破坏缓存前缀。

核心 Takeaways

- 命中率跨越：旧框架 84.26% → 新框架 97.11% (+12.9 个百分点)，末轮上下文 **512,028 tokens**。
- 成本跨越：同样 512K 目标、56 轮左右的对话，估算成本 **\$0.58 → \$0.12**，降幅约 79%。
- 前缀稳定性：切换轮命中率 97.24% 与稳定轮 97.03% 基本持平，追加式 persona 上下文消除了切角色导致的缓存失效。

本文档回答的问题

1. 在「必须调用工具 + 中途切换角色 + 模拟真实用户」的条件下，512K 上下文能维持多高的缓存命中率？
2. 追加式 persona 上下文相比旧「换入 systemPrompt」方案，对缓存前缀和成本有多大影响？
3. 命中率与成本数据是否可复现、可审计 (telemetry 请求级归属)？

背景与问题定义

PiLore 是一个 AI 编码教育 agent 核心：LLM → 工具调用 → 远程沙箱执行 → 基于真实输出的讲解。它面向长会话教学，每一轮都会把完整历史重新发给模型，因此上下文缓存命中率直接决定单会话成本量级。

为什么缓存命中率是关键指标

DeepSeek(deepseek-v4-flash)按前缀缓存计费：请求中与上一请求相同的 prompt 前缀按 **cacheRead** 超低价计费，未命中的新增部分按正常 **input** 计费。在 512K 上下文下，每轮重发完整历史，若前缀稳定则绝大多数 token 命中缓存；若前缀被打断(如 systemPrompt 变化)，整段历史重新计费，成本成倍上升。旧框架下未命中 input 累计达 **368 万 tokens**，正是前缀被打断的代价。

核心问题

教学会话天然包含三类「前缀破坏因素」：工具链多回合(write_file / run_code / read_file)、角色切换(Oris / Feynman / Socrates / 自动路由)、教学进度更新(update_teaching)。旧实现把角色方法论整体换入 systemPrompt，切换一次前缀即失效；新实现改为追加式 **persona** 上下文——方法论作为内部消息与下一条用户消息合并，systemPrompt 全程固定，前缀保持稳定。

「切换角色就该掉命中率」并不是必然。前缀缓存只认「请求前缀是否与上次相同」，与语义是否变化无关——把变化移出 systemPrompt、移进消息流，就能在保留角色切换的前提下保住前缀。

衡量标准

维度	旧框架(换入式)	新框架(追加式)	差距
累计缓存命中率	84.26%	97.11%	+12.9 pp
未命中 input(累计)	3,681,581	486,880	-87%
估算成本(512K 目标)	\$0.58	\$0.12	-79%
HTTP 重试次数	0	0	持平

方法与发现

测试不使用纯文字对话，而是完整启用 PiLore Agent 的真实行为：调用工具、中途切换角色、逐轮累积上下文至 512K，并通过请求级 telemetry 记录每次逻辑调用与真实 HTTP 请求的命中数据。

研究方法

驱动 createEduSession (Agent 核心会话层)，注入默认老师集合与真实沙箱 `http://192.168.172.134:1313`。每轮模拟一种真实用法：写代码跑码(`write_file + run_code`)、贴学习材料要求讲解、概念问答、贴 bug 代码要求修复并验证。每 3 轮按 Oris → Feynman → Socrates → 自动路由 的节奏切换角色；`maxTurns` 设为 6 给工具链留足回合。上下文度量口径为「最后一回合 LLM 调用的 `input + cacheRead + cacheWrite`」，达到 512K 目标即停。

```
session = createEduSession({
  models, providerId: "deepseek", modelId: "deepseek-v4-flash",
  thinkingLevel: "off", personas,           # Oris / Feynman / Socrates
  llmTelemetry: { onEvent },                # 请求级归属:logicalRequestId /
callIndex
  maxTurns: 6,                             # 工具链护栏
})
for r in range(rounds):
  session.setPersona(rotation(r % 3)) # 中途切换角色
  await session.prompt(buildUserMessage(r), onEvent)
```

三轮测试总览

三次测试的对话模式与工具调用保持一致，差异在框架版本与运行环境：第一次为旧「换入式」实现，后两次为「追加式 persona 上下文」实现，其中第三次为换用新 API key 后的复测，用于验证数据可复现。三次均 0 次 HTTP 重试。

轮次	框架	对话轮数	末轮上下文	累计命中率	未命中 input	估算成本
第 1 次	旧(换入式 systemPrompt)	56	526,147	84.26%	3,681,581	\$0.58
第 2 次	新(追加式 persona 上下文)	56	549,194	98.30%	519,607	\$0.17
第 3 次	新(换新 key 复测)	55	512,028	97.11%	486,880	\$0.12

关键发现

发现 1: 追加式 persona 上下文把命中率抬升 12.9 个百分点

旧框架下切换角色 = 换入新 systemPrompt = 前缀失效, 单轮未命中 input 常飙至 10 万以上(如 114,788 与 92,228)。新框架 systemPrompt 全程哈希不变, 切换只追加上下文, 命中率稳定在 97% 以上。第 3 次复测 (97.11%) 与第 2 次 (98.30%) 基本一致, 证明提升来自框架而非单次运行。

发现 2: 角色切换不再破坏缓存前缀

新框架第 3 次测试中, 切换轮命中率 97.24%、稳定轮 97.03%, 两者几乎持平; 末轮 `commonPrefixMessages = 136`, 即最后一次请求与上次共享 136 条完整历史消息。旧框架则无此保证——每次切换后整段历史重新计费。

前缀缓存只关心「这段前缀我上次是否见过」。把变化从 systemPrompt 挪进消息流, 角色切换就从「缓存杀手」变成了「一次普通的新增消息」。

— 本次测试的核心观察

发现 3: 请求级 telemetry 可审计, 计数自治

第 3 次测试共 66 次逻辑调用、66 个 assistant 成功消息、66 次 HTTP 请求、0 重试, 三者完全一致; 报告按 `logicalRequestId / callIndex` 归属每个调用, 可逐请求核对命中与成本。第 2 次为 109 次调用(工具链更长), 同样 0 重试。

发现 4: 工具调用模式在两次新框架测试中略有差异

工具	第 2 次	第 3 次
write_file	14	7
run_code	14	8
read_file	3	1
update_teaching	0	1
合计	31	17

04 · CONCLUSIONS

| 结论与建议

追加式 persona 上下文是这次命中率与成本改善的核心来源，且在第 2、3 次测试中可复现；512K 真实教学会话下的累计命中率稳定在 97% 以上。

核心结论

1. 追加式 persona 上下文将累计缓存命中率从 84.26% 提升至 97.11%，估算成本从 \$0.58 降至 \$0.12(降 79%)。
2. 角色切换不再破坏缓存前缀：切换轮命中率 97.24% 与稳定轮 97.03% 持平，末轮前缀共享 136 条历史消息。
3. 数据可复现、可审计：换新 key 复测命中率 97.11%(前次 98.30%)，66 次逻辑调用 = 66 次 HTTP = 0 重试，请求级归属一致。

下一步建议

保持「systemPrompt 固定 + 方法论追加进消息流」的架构不变；后续优化可关注工具链回合数（第 2 次 109 次调用 vs 第 3 次 66 次），因为每个工具回合都是一次完整历史重发，减少不必要的工具调用能进一步压低成本。DeepSeek 控制台的命中率与账单可作为服务端侧核对依据。

CALL TO ACTION

如果只需记住一件事：把任何会变化的上下文（角色、进度、指令）移出 systemPrompt、以追加式内部消息进入历史，前缀缓存就能在真实多轮使用中保持 97% 以上的命中率。

| 附录

A. 数据来源

- reports/cache-realistic-2026-08-10T14-31-22-668Z.json (第 1 次, 旧框架)
- reports/cache-realistic-2026-08-10T15-41-28-797Z.json (第 2 次, 新框架)
- reports/cache-realistic-2026-08-11T03-04-46-274Z.json (第 3 次, 新框架 · 换新 key 复测)

B. 术语表

前缀缓存(prefix caching): DeepSeek 服务端对请求前缀的自动缓存; 命中的 token 按 cacheRead 超低价计费。

cacheRead / cacheWrite / input: pi-ai usage 字段, 分别对应缓存命中、缓存写入、未命中新增部分。

追加式 persona 上下文: 方法论作为内部消息与下一条用户消息合并, systemPrompt 全程固定。

logicalRequestId / callIndex: telemetry 中逻辑调用的唯一标识与全局递增序号, 用于请求级归属。

commonPrefixMessages: 当前请求与上次请求共享的完整历史消息条数。

C. 测试条件

模型: deepseek-v4-flash · thinking off; **执行后端**: 192.168.172.134:1313(真实沙箱); **上下文目标**: ~512K tokens; **单轮护栏**: maxTurns=6; **角色节奏**: 每 3 轮切换一次。